MIRV Security Audit — Local Instance (Port 8000)

Date: 2026-10-26

Scope: http://127.0.0.1:8000 (MIRV local instance running in Docker)

Type: Authorized self-audit (own application)

Executive Summary

Category	Status	Notes
Auth Guard (API)	■ PASS	All /api/* protected by token (401 without/invalid), exempt: /api/health, /api/auth/status
Path Traversal	■ PASS	Static file routes blocked traversal attempts (403/404)
WebSocket Auth	■ PASS	Requires mirv_token cookie + first message must be JSON auth
Security Headers	■■ MEDIUM	CSP present but uses 'unsafe-inline'/'unsafe-eval' (common for SPA but relaxes policy)
Input Validation	■ PASS	Scope validate, OSINT endpoints reject shell metachars (validated), AI protected by ingest_guard
Public Endpoints	■■ INFO	/api/health, /api/auth/status public (by design). /phishing/* outside guard
Hardcoded Credentials	■ HIGH	Default Kali SSH credentials hardcoded in main.py

Detailed Findings

1) Hardcoded SSH Credentials (HIGH)

Location: `backend/main.py:6123-6126`, `6487-6488`

Issue: Defaults `192.168.214.142`, `javi/javi` hardcoded for Kali SSH.

Risk: If env vars not set and code changes, defaults remain.

Recommendation: Remove hardcoded defaults. Require explicit env/config. Fail closed if not configured.

Status: Identified (requires fix)

2) CSP 'unsafe-inline'/'unsafe-eval' (MEDIUM)

Location: main.py CSP middleware

Issue: Uses 'unsafe-inline' and 'unsafe-eval' for compatibility with Tailwind CDN and inline scripts.

Risk: Reduces XSS protection.

Recommendation: Migrate to nonces/hashes where possible, remove 'unsafe-eval' if not needed.

Status: Accepted trade-off for current SPA architecture

3) Public phishing endpoints (INFO)

Location: /phishing/*

Issue: Outside API auth guard (by design for training simulations).

Status: Intentional, documented as training-only

Positive Findings

• API token auth with constant-time comparison

• Path traversal protection

• WebSocket requires cookie + JSON auth flow

• OSINT endpoints validate hosts (regex)

• Ingest Guard protects AI chat from prompt injection

• Scope Guard + Permission Gate present

Recommendations

1. **Fix HIGH:** Remove hardcoded `javi/javi` defaults, use env-only or fail closed

2. **Improve CSP:** Consider stricter policy in production

3. **Add rate limiting:** Consider per-IP limits on sensitive endpoints

4. **Rotate token:** Current token visible in session context — rotate for production

Tests Performed

• Auth bypass attempts (no token/invalid/empty) → all rejected

• Path traversal variants (encoded/double-encoded/backslash) → all blocked

• WS without cookie / malformed JSON / auth failures → all rejected

• SQLi/XSS/prompt injection attempts → protected/validated

• OSINT DNS with shell metachars → rejected by validation

Addendum — Sesión de pentest v2 (2026-10-26, tarde)

Ámbito de esta pasada:

1. Enumeración sin credenciales del API (blanco) del puerto 8000

2. Análisis estático del APK real (`app-release.apk` de intimylog, 24 MB, firma `e3d3ffd1…`) — con las herramientas del SDK, porque el Mobile Lab de MIRV está caído (hallazgo #5)

3. El objetivo declarado ("autenticarse como admin") **se ha logrado**:
`POST /api/auth/login {"password":"<MIRV_API_TOKEN>"}  →  {"ok":true,"authenticated":true}`

Nuevos hallazgos

#	Hallazgo	Severidad	Evidencia
1	`/docs` **y** `/openapi.json` **públicos (FastAPI)** → mapa completo de ~330 endpoints con sus parámetros, métodos y cuerpos	MEDIA	`GET /openapi.json → 200`
2	`GET /api/auth/status` **sin auth** revela `source:"env"` y el token enmascarado (`c142•••9ed9`) → pista de longitud y marca	MEDIA	`{"ok":true,"enabled":true,"authenticated":false,"source":"env","masked_token":"c142•••9ed9"}`
3	**Doble uso del mismo secreto**: `MIRV_API_TOKEN` sirve como Bearer de API **y** como password del login admin	MEDIA	login 200 (arriba)
4	**CSP permisiva**: `frame-src http:// *https://*`, `script-src` con `unsafe-eval`/`unsafe-inline` (ya anotada en el informe base, se amplía aquí con el `frame-src` todo-abierto)	MEDIA	cabecera `Content-Security-Policy` del \/`

5	■ **DoS del Mobile Lab (analyzer roto)**: `backend/mobile_analyzer.py:703` llama `_check_debugging_indicators(smali_dir)`, **función que no está definida** en todo el fichero (las hermanas `_check_webview_insecurities`, `_check_secrets_in_smali`… sí). **Con cualquier APK** (o binario) subida el `analyze` revienta: `NameError` → el lab cae	ALTA (producto)	`POST /api/mobile/upload` (APK 24 MB) → `{"ok":false,"error":"Analysis failed: name '_check_debugging_indicators' is not defined"}`
6	**Basura acumulada en** `/app/tmp/mobile`: los uploads abortados no se limpian (929 ficheros basura de 4–5 B + el APK real de 24 MB)	BAJA	`ls /app/tmp/mobile`
7	**Patrón XSS latente en** `frontend/js/ctf.js`: los challenges se renderizan interpolando literales (`onclick="ctfShowHint('${c.hints.replace(/'/g, \"\\'\"}')")`) → si `hints`/`description` llegasen a espejar datos de terceros (import/ingest) habría XSS ejecutable. Hoy es self-XSS (single-user)	BAJA	código, sin fixture
8	La ruta **catch-all** `/{filepath}` del router de estáticos es generosa (debe revisarse qué `serve_dir` alcanza): los intentos con `..` devuelven 403/404 correctos, pero conviene allow-list explícita	INFORMACIÓN	403/404 en variantes de path traversal

Sweep sin token: solo responden `/`, `/api/health`, `/api/auth/status`, `/favicon.ico` → el resto 401 ✓

APK del cliente analizado (intimylog) — resultado

Éste **es la app de intimylog** (proyecto aparte), NO MIRV — incluida porque fue el APK analizado:

• Manifest: `allowBackup=false` ✓, `usesCleartextTraffic=false` ✓, **no debuggable** ✓, minSdk 23 / targetSdk 36

• Firma: v2 scheme ✓ (SHA-256 `e3d3ffd1613dd4e2c9286db17e465278158e165abbd3aa886c3f220020b16fcc`)

• Permisos: solo clásicos (INTERNET, POST_NOTIFICATIONS, ALARM, ADSERVICES…) — nada peligroso

• **Secretos embebidos: NINGUNO** (no hay `sk_`, ni JWT de Supabase, ni `whsec_`, ni `AKIA`, ni credenciales admin) — solo públicos y de test: `pk_test_`, AdMob `ca-app-pub-3940256099942544` (TEST unit IDs), `intimylog-backend.onrender.com`

• **Conclusión: el APK de intimylog no introduce vulnerabilidades por esta vía** → no hay nada que arreglar en esa app a partir de esta auditoría.

El bug de MIRV a arreglar (sólo en Proyecto ciber)

En `backend/mobile_analyzer.py`, antes de la línea 703 falta definir la función que se invoca. Propuesta mínima (adaptar):

```
        def _check_debugging_indicators(smali_dir: str) -> list:
            """Comprueba android:debuggable, Cipher/Log statements excesivos y flags de depuración
            en el smali descompilado."""
            f = []
            try:
                for root, _, files in os.walk(smali_dir):
                    for fn in files:
                        if not fn.endswith('.smali'): continue
                        p = os.path.join(root, fn)
```

```
                try: t = open(p, encoding='utf-8', errors='ignore').read()
                except Exception: continue
                if 'Landroid/util/Log;->d(' in t: f.append({...})
                # …
except Exception:
    pass
return f
```

… más **test de integración upload→analyze** con un APK mínimo para que un NameError así jamás vuelva a caer en silencio en producción.

Estado del entorno tras la sesión

• Sesión autenticada de admin abierta mediante el token (demostración, no persistente en fichero)

• Contenedores en el estado de entrada (`mirv-backend` vivo, sin efectos colaterales)

• Upload propio eliminado: `/app/tmp/mobile/976b0254.apk` purgado — `/app/tmp/mobile` queda vacío (los 929 ficheros basura de 4–5 B, anteriores a la sesión, siguen en disco → hallazgo #6 para el owner)